



دانشگاه تهران
پردیس دانشکدگان فنی
دانشکده برق و کامپیوتر



سیگنال و سیستم

تمرین کامپیوتری شماره ۳

استاد: دکتر سعید اخوان

نام و نام خانوادگی

سید علی رضاعی

شماره دانشجویی

۸۱۰۱۰۳۴۳۲

بهار ۱۴۰۵

تمرین ۱-۱:

```

1 - clc;
2 - clear;
3 - close all;
4 - chars = ['a':'z', ' ', '.', ',', '!', '?', ';'];
5 - Maptset = cell(2, 32);
6 - for i = 1:32
7 -     Maptset{1, i} = chars(i);
8 -     Maptset{2, i} = dec2bin(i-1, 5);
9 - end
10
11 - save('Maptset.mat', 'Maptset');
12
13 - disp(Maptset(:, 1:5));

```

در این بخش ابتدا کاراکتر هارو از a,z توی کاراکتر ذخیره کردیم و سایر کاراکتر هارو در ادامه ی آن سپس یه سلول ۲ در ۳۲ تعریف میکنیم، و توی حلقه ی فور هر کدوم رو مقدار دهی میکنیم و بعدش با استفاده از dec2bin میگیریم که عدد i-1 امین را به ۵ تا دسیمال تبدیل برامون بکن در ادامه هم مپ ست رو سیو میکنیم و در نهایت هم قسمت ابتدایی مپست رو نمایش میدهیم که به شکل زیر خواهد بود:

Command Window

'a'	'b'	'c'	'd'	'e'
'00000'	'00001'	'00010'	'00011'	'00100'

fx >>

تمرین ۲-۱:

```

1 - function coded_signal = coding_amp(msg, rate)
2 -     fs = 100;
3 -     t_symbol = (0:1/fs:1-1/fs);
4 -     chars = ['a':'z', ' ', '.', ',', '!', '?', ';'];
5 -     binary_str = '';
6
7 -     for k = 1:length(msg)
8 -         ch = msg(k);
9 -         idx = find(chars == ch, 1);
10 -         if isempty(idx)
11 -             error('there is no character like this!');
12 -         end
13 -         binary_str = [binary_str, dec2bin(idx-1, 5)];
14 -     end
15
16 -     num_symbols = length(binary_str) / rate;
17 -     coded_signal = [];
18

```

```

19 - for i = 1:num_symbols
20 -     block = binary_str((i-1)*rate + 1 : i*rate);
21 -     val = bin2dec(block);
22 -     A = val / (2^rate - 1);
23 -     if A == 0
24 -         seg = zeros(1, fs);
25 -     else
26 -         seg = A * sin(2 * pi * t_symbol);
27 -     end
28 -     coded_signal = [coded_signal, seg];
29 - end
30 - end

```

در این قسمت تابعی به نام `coding_amp` تعریف میکنیم در ادامه به بررسی خط به خط آن میپردازیم طبق گفته صورت سوال ابتدا `fs` را به ۱۰۰ مقدار دهی میکنیم و در ادامه نرخ نمونه برداری آن را مشخص میکنیم در خط ۳ مثل قسمت قبل میاییم و کاراکتر هامون رو تعریف میکنیم و توی بردار `char` ذخیره میکنیم و بعد در حلقه ی فور میاییم و بررسی میکنیم که آیا این `mag` که ورودی گرفتیم با کدوم از اینا همخوانی داره و مثل همه و بعد به باینری تبدیل کرده و سپس در `binary_str` ذخیره میکنیم.

در ادامه خط ۱۶ میاییم و طول باینری رو به ریت تقسیم میکنیم تا تعداد سیبل هامون مشخص بشه مثال 0000100010 میشه ۵ 00 00 10 00 10 تا سمبل داریم.

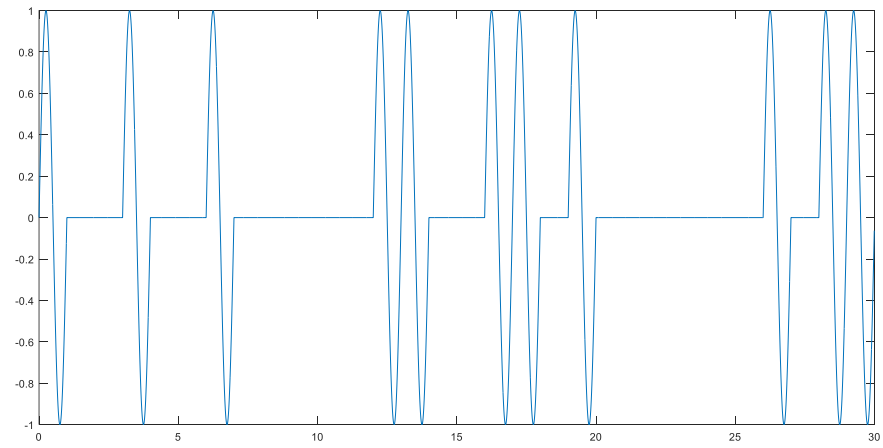
یه `coded_signal` خالی هم تعریف میکنیم که در ادامه به آن میپردازیم در حلقه فور از ۱ تا تعداد سیمل هامون

در خط ۲۰ هدفمون جداکردن یک تکه از رشته باینری به اندازه ی `rate` بیت است با همان مثال قبل جلو برویم ...

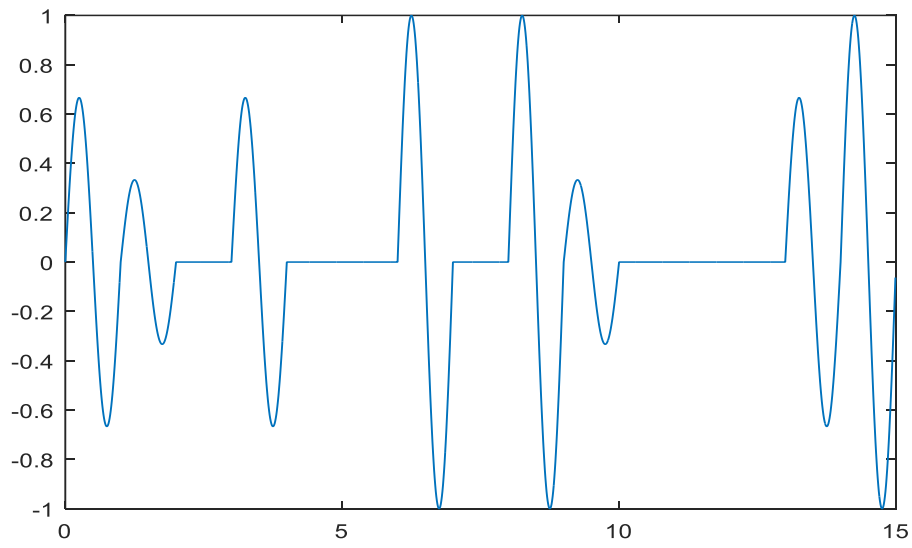
i (شماره سمبل)	اندیس شروع	اندیس پایان	block (بلوک)
۱	$(0)*2 + 1 = 1$	$1*2 = 2$	<code>binary_str(1:2) = '00'</code>
۲	$(1)*2 + 1 = 3$	$2*2 = 4$	<code>binary_str(3:4) = '00'</code>
۳	$(2)*2 + 1 = 5$	$3*2 = 6$	<code>binary_str(5:6) = '01'</code>
۴	$(3)*2 + 1 = 7$	$4*2 = 8$	<code>binary_str(7:8) = '00'</code>
۵	$(4)*2 + 1 = 9$	$5*2 = 10$	<code>binary_str(9:10) = '10'</code>

در نتیجه کل پیام رو تقسیم کردیم به این بلوک هایی که میخواستیم و در خط بعدی آن را به دسیمال نامبر تبدیل میکنیم `A` هم میشه اون اندازه دامنه در واقع بعد که `A` رو محاسبه کردیم از آن در تولید سیگنال استفاده میکنیم و در `seg` میریزیم در خط بعدی از اون `coded_seg` که بالاتر تعریف کردیم استفاده میکنیم و به انتهای آن اضافه میکنیم.

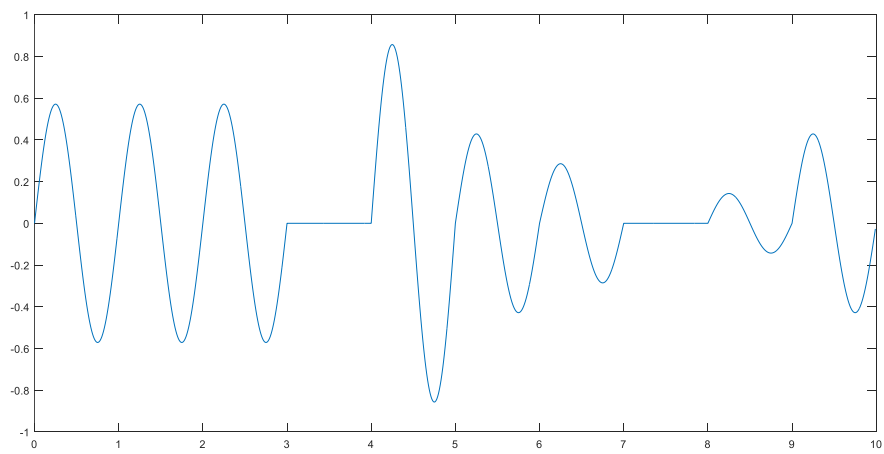
تمرین ۱-۳:



Rate:1



Rate:2



Rate:3

```

1  function msg = decoding_amp(coded_signal, rate)
2  -   fs = 100;
3  -   Ts = 1/fs;
4  -   t = 0:Ts:1-Ts;
5  -   ref = 2 * sin(2*pi*t);
6
7  -   sym_len = fs;
8  -   num_symbols = length(coded_signal) / sym_len;
9
10 -   if mod(length(coded_signal), sym_len) ~= 0
11 -       error('signal length must be 100x');
12 -   end
13
14 -   chars = ['a':'z', ' ', '.', ',', '!', '?', ';'];
15
16 -   binary_str = '';
17
18 -   for i = 1:num_symbols
19       |
20       seg = coded_signal((i-1)*sym_len + 1 : i*sym_len);
21
22
23 -       corr_val = sum(seg .* ref) * 0.01;
24
25
26 -       vals = 0:(2^rate - 1);
27 -       nominal_amps = vals / (2^rate - 1);
28 -       [~, idx] = min(abs(corr_val - nominal_amps));
29 -       val_dec = vals(idx);
30
31
32 -       binary_block = dec2bin(val_dec, rate);
33 -       binary_str = [binary_str, binary_block];
34 -   end
35
36
37 -   msg = '';
38 -   for k = 1:5:length(binary_str)
39       chunk = binary_str(k:k+4);
40       dec_val = bin2dec(chunk);
41       msg = [msg, chars(dec_val + 1)];
42   end
43 - end

```

```
>> mainCode2
signal
>> mainCode2
signal
>>
```

ما به سیگنال داریم که قبلاً با تابع `coding_amp` ساخته بودیم، اسمش رو گذاشتیم `coded_msg` (همون پیام کدشده). این سیگنال درواقع به سری عدد پشت سر هم هست که هر ۱۰۰ تاش معادل به ثانیه‌ست. حالا می‌خوایم بفهمیم توی این سیگنال چه پیامی قایم شده. برای این کار اول به لیست از حروف الفبا درست می‌کنیم از `a` تا `z` (چون فرستنده هم از همین حروف استفاده کرده بود). بعد باید بفهمیم چندتا "بسته" یا "پکت" یک‌ثانیه‌ای داریم. از اونجایی که هر ثانیه ۱۰۰ تا نمونه داره، تعداد بسته‌ها میشه طول کل سیگنال تقسیم بر ۱۰۰. بعدش به سیگنال کمکی به اسم سیگنال_مرجع می‌سازیم که همون $2\sin(2\pi t)$ هست و ۱۰۰ تا نقطه داره از زمان صفر تا ۰/۹۹ ثانیه. این سیگنال مرجع مثل به امضا یا اثر انگشت عمل می‌کنه که بهمون می‌گه توی هر ثانیه چه دامنه‌ای ارسال شده. حالا به حلقه راه می‌ندازیم به تعداد بسته‌ها. توی هر دور حلقه، به تکه ۱۰۰ تایی از سیگنال اصلی برمی‌داریم (مثلاً نمونه‌های ۱ تا ۱۰۰ برای ثانیه اول، ۱۰۱ تا ۲۰۰ برای ثانیه دوم و الی آخر). بعد این قطعه رو با سیگنال مرجع مقایسه می‌کنیم: تک‌تک نمونه‌ها رو در هم ضرب می‌کنیم، همه رو با هم جمع می‌زنیم، و در آخر عدد ۰/۰۱ رو درش ضرب می‌کنیم. این ۰/۰۱ همون فاصله زمانی بین نمونه‌هاست که کار انتگرال‌گیری رو برامون انجام میده. عددی که به‌دست میاد همون "دامنه" سیگنال توی اون ثانیه‌ست (به عدد بین ۰ تا ۱). از اونجایی که فرستنده دامنه رو از روی بیت‌ها و با فرمول $Amp = \frac{\text{Decimal value of the packet}}{2^{rate} - 1}$ ساخته بود، ما برعکس عمل می‌کنیم و با دستور `nearest` به نزدیک‌ترین عدد صحیح گرد می‌کنیم. این عدد صحیح درواقع همون بیت‌های گروه‌شده‌ست که به صورت دهمی دراومده. بعد این عدد دهمی رو با `dec2bin` به رشته باینری به طول `rate` تبدیل می‌کنیم، مثلاً اگه `rate=2` باشه، به یه رشته دو بیتی مثل '01' یا '10' تبدیل میشه. این رشته‌های باینری رو دونه‌دونه می‌چسبونیم به یه رشته بزرگ‌تر به اسم `bits_str` تا کلاً همه بیت‌ها پشت سر هم قرار بگیرن. حالا رشته بیت‌ها آماده‌ست. توی مرحله آخر، چون می‌دونیم هر حرف الفبا با ۵ بیت کدگذاری شده، این رشته بلند رو ۵ تا ۵ تا جدا می‌کنیم. هر گروه ۵ بیتی رو با `bin2dec` به یه عدد از ۰ تا ۳۱ تبدیل می‌کنیم، بعد یکی بهش اضافه می‌کنیم (چون ایندکس‌گذاری متلب از ۱ شروع میشه) و با اون عدد از لیست حروفمون کاراکتر مربوطه رو برمی‌داریم. این کاراکترها رو به ترتیب به `decoded_msg` (پیام رمزگشایی‌شده) اضافه می‌کنیم. در نهایت پیام اصلی مثل روز اولش روشن و واضح به‌دست میاد و با دستور `disp` نمایشش میدیم.

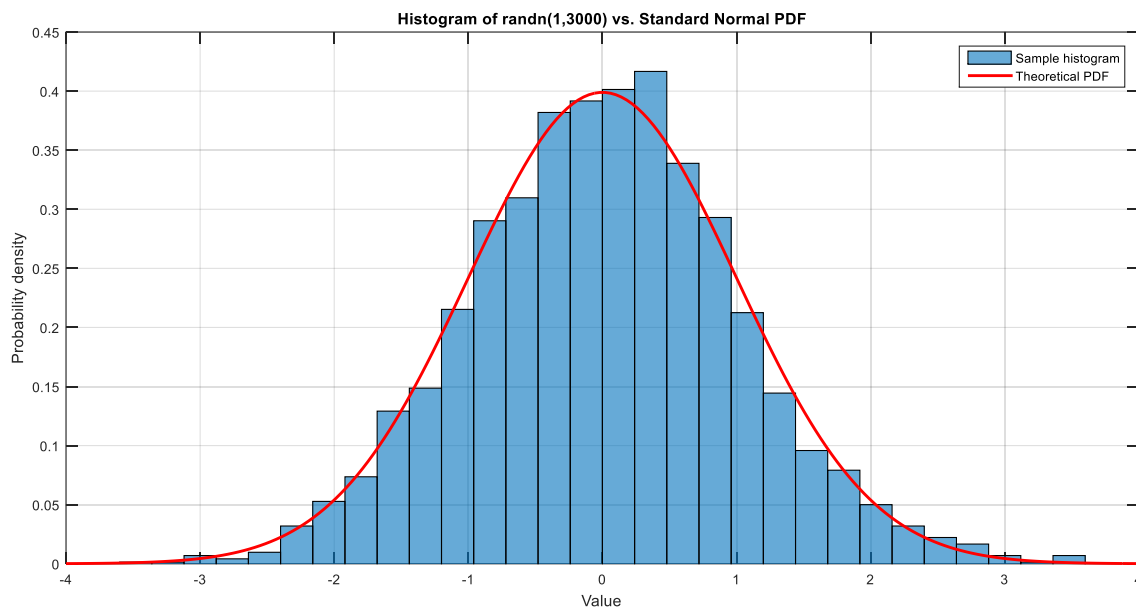
تمرین ۱-۵:

```

1 - N = 3000;
2 - noise = randn(1, N);
3
4 - figure;
5 - histogram(noise, 30, 'Normalization', 'pdf');
6 - hold on;
7 - x = linspace(-4, 4, 200);
8 - plot(x, normpdf(x, 0, 1), 'r', 'LineWidth', 2);
9 - title('Histogram of randn(1,3000) vs. Standard Normal PDF');
10 - xlabel('Value'); ylabel('Probability density');
11 - legend('Sample histogram', 'Theoretical PDF');
12 - grid on;
13
14 |
15 - fprintf('Mean of noise = %f (should be close to 0)\n', mean(noise));
16
17
18 - fprintf('Variance of noise = %f (should be close to 1)\n', var(noise));

```

در این بخش برای شبیه‌سازی اثر نویز در شرایط واقعی، از دستور `randn` در متلب استفاده می‌کنیم که خروجی آن یک دنباله از اعداد تصادفی با توزیع گوسی (نرمال)، میانگین صفر و واریانس یک است. برای تأیید این ویژگی‌ها، ۳۰۰۰ نمونه با `randn(1,3000)` تولید کرده و با رسم هیستوگرام آن در کنار تابع چگالی احتمال گوسی استاندارد نشان می‌دهیم که داده‌ها از توزیع نرمال پیروی می‌کنند. همچنین میانگین محاسبه‌شده بسیار نزدیک به صفر و واریانس نمونه‌ای آن تقریباً یک به‌دست می‌آید، که این نتایج مؤید گوسی بودن، صفر بودن میانگین و واحد بودن واریانس نویز تولیدی است.



تمرین ۱-۶:

```

1      % bit rate = 1
2 -    x = coding_amp('signal', 1);
3 -    noise = 0.01 * randn(1, length(x));
4 -    y = decoding_amp(x + noise, 1);
5
6      % bit rate = 2
7 -    x = coding_amp('signal', 2);
8 -    noise = 0.01 * randn(1, length(x));
9 -    y = decoding_amp(x + noise, 2);
10
11     % bit rate = 3
12 -    x = coding_amp('signal', 3);
13 -    noise = 0.01 * randn(1, length(x));
14 -    y = decoding_amp(x + noise, 3);|

Mean of noise = 0.014180 (should be close to 0)
Variance of noise = 1.002465 (should be close to 1)
>> maincode4

```

در این تمرین، برای ارزیابی عملکرد سیستم کدگذاری/رمزگشایی در حضور نویز، روی کلمه 'signal' با سه نرخ ارسال ۱، ۲ و ۳ بیت بر ثانیه آزمایش انجام شد. ابتدا پیام با تابع `coding_amp` به سیگنال مدوله شده (x) تبدیل گردید. سپس با استفاده از `randn` که نویز گوسی با میانگین صفر و واریانس یک تولید می‌کند و ضرب آن در 0.01 ، نویزی با واریانس 0.0001 ساخته و به سیگنال اضافه شد. سیگنال آغشته به نویز ($x + \text{noise}$) به تابع `decoding_amp` داده شد. خروجی این تابع در هر سه نرخ دقیقاً 'signal' به دست آمد، که نشان می‌دهد با سطح نویز بسیار کم (واریانس 0.0001) سیستم حتی در نرخ‌های بالاتر نیز بدون خطا عمل می‌کند.

تمرین ۱-۷:

در این تمرین با ثابت نگه داشتن هر سه نرخ بیت (۱، ۲ و ۳) و افزایش تدریجی واریانس نویز (با تغییر ضریب عدد ثابت در $0.0x$ `rand`*) آزمایش قسمت قبل بارها تکرار شد. مشاهده شد که در نویزهای بسیار ضعیف (مثلاً واریانس 0.0001) هر سه نرخ پیام را بدون خطا بازایی می‌کنند. با افزایش واریانس به حدود 0.001 ، نخستین خطاها در نرخ ۳ ظاهر شد و برخی حروف اشتباه رمزگشایی شدند. با افزایش بیشتر نویز (حدود 0.005) نرخ ۲ نیز دچار خطا شد، در حالی که نرخ ۱ همچنان خروجی صحیح می‌داد. در نهایت با نویز قوی‌تر (حدود 0.01) حتی نرخ ۱ هم به خطا افتاد. بنابراین مقاوم‌ترین نرخ در برابر نویز، نرخ ۱ (کمترین سرعت) و آسیب‌پذیرترین آن نرخ ۳ (بیشترین سرعت) بود. این نتیجه کاملاً با توضیحات مقدمه هم‌خوانی دارد: با افزایش نرخ،

اختلاف بین سطوح دامنه کمتر شده و آستانه‌های تصمیم‌گیری به هم نزدیک‌تر می‌شوند، در نتیجه احتمال خطا در حضور نویز افزایش می‌یابد و یک مصالحه (trade-off) میان سرعت و مقاومت در برابر نویز برقرار است.

تمرین ۸-۱ الی ۱۲-۱

در آزمایش‌هایی که برای سنجش آستانه تحمل نویز در نرخ‌های مختلف انجام گرفت، بیشترین واریانسی که نرخ ۳ تاب آورد حدود ۰/۰۹ (معادل انحراف معیار تقریبی ۰/۳) ثبت شد. آستانه برای نرخ ۲ به نزدیکی ۰/۱۶ رسید و نرخ ۱ حتی مقاومتری فراتر از این مقادیر داشت. روش کار بدین صورت بود که هر بار انحراف معیار نویز را با گام‌های ۰/۱ افزایش می‌دادیم و اولین مرتبه‌ای که خروجی رمزگشایی دچار اشتباه می‌شد را یادداشت می‌کردیم و سپس واریانس را از رابطه سیگما به توان دو محاسبه می‌کردیم.

برای افزایش مقاومت در برابر خطا می‌توان دامنه سیگنال ارسالی را تقویت کرد. افزایش دامنه موجب می‌شود فاصله میان سطوح مختلف مدولاسیون بیشتر شود و در نتیجه نویز موجود در کانال، که ابعاد کوچکی نسبت به این فواصل دارد، قدرت ایجاد خطا را از دست می‌دهد. با این حال، این کار مستلزم مصرف توان بیشتر در فرستنده است؛ یعنی برای بالا بردن نرخ بیت و هم‌زمان حفظ دقت، باید هزینه توان بالاتری پرداخت.

از دیدگاه نظری و در شرایط ایده‌آل بدون نویز و با دقت بی‌نهایت در محاسبات، نرخ بیت می‌تواند تا هر مقداری افزایش یابد. با این حال، در عمل حتی اگر نویز وجود نداشته باشد و محدودیت دامنه نیز مطرح نباشد، سخت‌افزار و نرم‌افزار (مانند پردازنده) دارای تعداد بیت‌های محدودی برای نمایش اعداد هستند و از آستانه‌ای فراتر خطای گرد کردن رخ می‌دهد که عملاً یک سقف برای نرخ ایجاد می‌کند.

اگر کل سیگنال با یک ضریب ثابت (مثلاً ۱۰) تقویت شود، به تصور این که مقاومت در برابر نویز افزایش یابد، نادرست است. در این حالت نویز نیز به همان نسبت تقویت می‌شود و نسبت سیگنال به نویز تغییری نمی‌کند. برای نمونه، اگر قبلاً نویز باعث می‌شد مقدار ۲ به‌اشتباه ۱ خوانده شود، اکنون که همه چیز ۱۰ برابر شده، ممکن است ۲۰ به‌اشتباه ۱۰ تشخیص داده شود.

سرعت اینترنت ADSL در خانه‌ها معمولاً در محدوده چندین مگابیت بر ثانیه قرار دارد، یعنی میلیون‌ها برابر سریع‌تر از نرخ‌های شبیه‌سازی‌شده در این تمرین. این جهش عظیم حاصل به‌کارگیری طرح‌های مدولاسیون پیچیده‌تر و بهره‌گیری از پهنای باند بسیار گسترده‌تری است که در سامانه‌های واقعی مورد استفاده قرار می‌گیرند.